Experiment 5: Sentiment Analysis Using Naive Bayes

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

# Small example dataset with positive and negative comments
comments = [
    "I love this product",
    "This is an excellent experience",
    "The service was wonderful",
    "I am very happy with the purchase",
    "The quality is amazing",
    "I really enjoyed this movie",
    "The product works perfectly",
    "Excellent customer support",
    "I am satisfied with the result",
    "This was a fantastic experience",
    "I hate this product",
    "This is a terrible experience",
    "The service was awful",
    "I am very disappointed",
    "The quality is poor",
    "I really disliked this movie",
    "The product does not work",
    "Very bad customer support",
    "I am unhappy with the result",
    "This was a horrible experience"
]

labels = [
    "Positive", "Positive", "Positive", "Positive", "Positive",
    "Positive", "Positive", "Positive", "Positive", "Positive",
    "Negative", "Negative", "Negative", "Negative", "Negative",
    "Negative", "Negative", "Negative", "Negative", "Negative"
]

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    comments,
    labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)
```

```
# Text vectorization + Naive Bayes classifier
model = Pipeline([
    ("vectorizer", CountVectorizer(lowercase=True)),
    ("classifier", MultinomialNB())
])

# Train the model
model.fit(X_train, y_train)

# Predict sentiment for test comments
y_pred = model.predict(X_test)

# Display test results
results = pd.DataFrame({
    "Comment": X_test,
    "Actual Sentiment": y_test,
    "Predicted Sentiment": y_pred
})

print("Classification Results:")
print(results.to_string(index=False))

accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy:.2%}")

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    labels=["Negative", "Positive"],
    zero_division=0
))

# Test a new comment
new_comment = "The product is excellent and I am happy"
prediction = model.predict([new_comment])[0]
print("New Comment:", new_comment)
print("Predicted Sentiment:", prediction)
```